

Cryptography

Complete Study Notes

Symmetric & Asymmetric Encryption · AES · Diffie-Hellman
Digital Signatures · Hash Functions · JWT · HTTPS

TOPICS COVERED

1. What is Cryptography?

2. Symmetric Encryption + DES + AES

3. Asymmetric Encryption + RSA

4. Diffie-Hellman Key Exchange

5. AES — How It Works (Rounds)

6. Digital Signatures — Auth & Integrity

7. Hash Functions — SHA Family

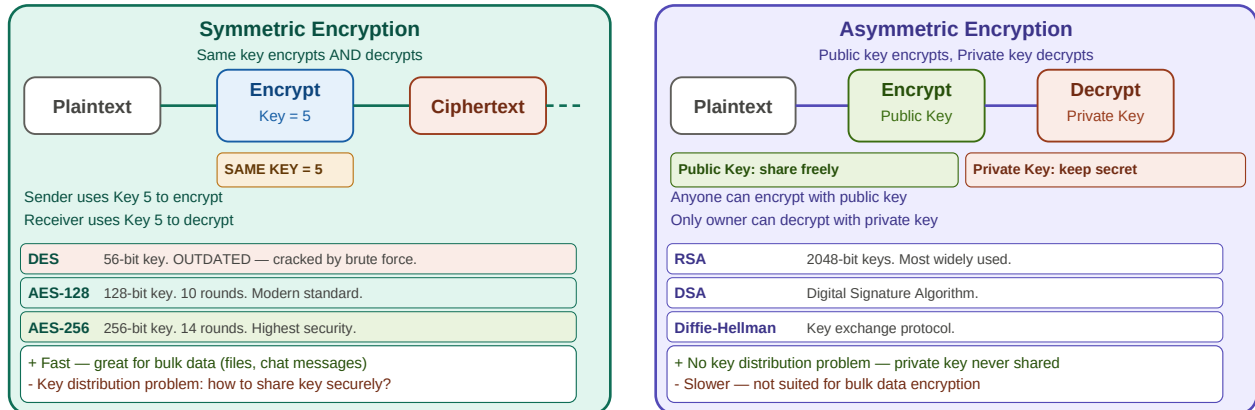
8. Real-World: HTTPS, JWT, E2E Chat

1. What is Cryptography?	3
2. Symmetric vs Asymmetric Encryption	3
3. AES — Advanced Encryption Standard	4
4. Diffie-Hellman Key Exchange	5
5. Digital Signatures — Authentication & Integrity	6
6. Algorithm Comparison & Real-World Usage	7
7. Quick Revision & Interview Questions	7

1 & 2. Cryptography — Symmetric vs Asymmetric

Cryptography is the science of securing information by transforming readable data (plaintext) into an unreadable form (ciphertext) using a key and an algorithm. It ensures **confidentiality** (only intended recipient can read), **integrity** (data not tampered with), and **authentication** (verifies identity).

Symmetric vs Asymmetric Encryption



How They Work Together in the Real World

- 1. Asymmetric** (Diffie-Hellman) securely exchanges a symmetric key over an insecure network.
 - 2. Symmetric** (AES) uses that shared key to encrypt all actual data at high speed.
- Example:** HTTPS — TLS uses DH to exchange AES session key, then AES encrypts all traffic.

Symmetric: same key encrypts+decrypts — fast, bulk data · Asymmetric: public/private key pair — slower, key exchange & signatures

Aspect	Symmetric	Asymmetric
Keys	Single shared secret key	Public key (encrypt) + Private key (decrypt)
Speed	Fast — low computation	Slow — large keys (2048-bit RSA)
Best for	Bulk data: files, chat messages, HTTPS traffic	Key exchange, digital signatures, small data
Key problem	Key distribution: how to share key securely?	No problem — private key is never shared
Algorithms	DES (outdated), AES-128/192/256	RSA, DSA, Diffie-Hellman, ECDH
Real example	AES encrypts WhatsApp messages	DH exchanges the AES key securely over HTTPS

The two types complement each other: asymmetric encryption (Diffie-Hellman) securely exchanges a symmetric key, then symmetric encryption (AES) uses that key to encrypt all actual data at speed.

DES (56-bit): obsolete — cracked by brute force. AES-128/192/256: modern standard, not yet cracked.

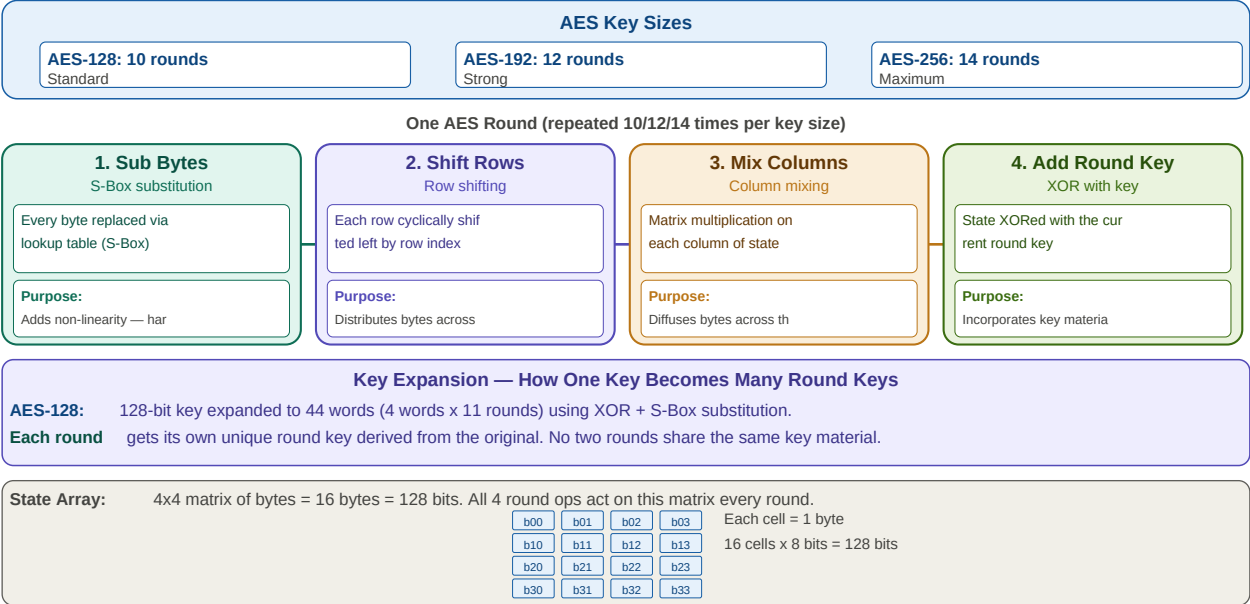
RSA-2048: the "2048-bit" refers to the key size. Larger key = harder to break = slower to compute.

HTTPS = TLS uses DH to exchange an AES session key, then AES encrypts all HTTP traffic in that session.

3. AES — Advanced Encryption Standard

AES is the modern symmetric encryption standard, used everywhere from HTTPS to WhatsApp to disk encryption. It operates on **128-bit data blocks** using a **4x4 matrix of bytes** called the State Array. Each encryption round applies 4 mathematical operations to transform the data.

AES (Advanced Encryption Standard) — How It Works

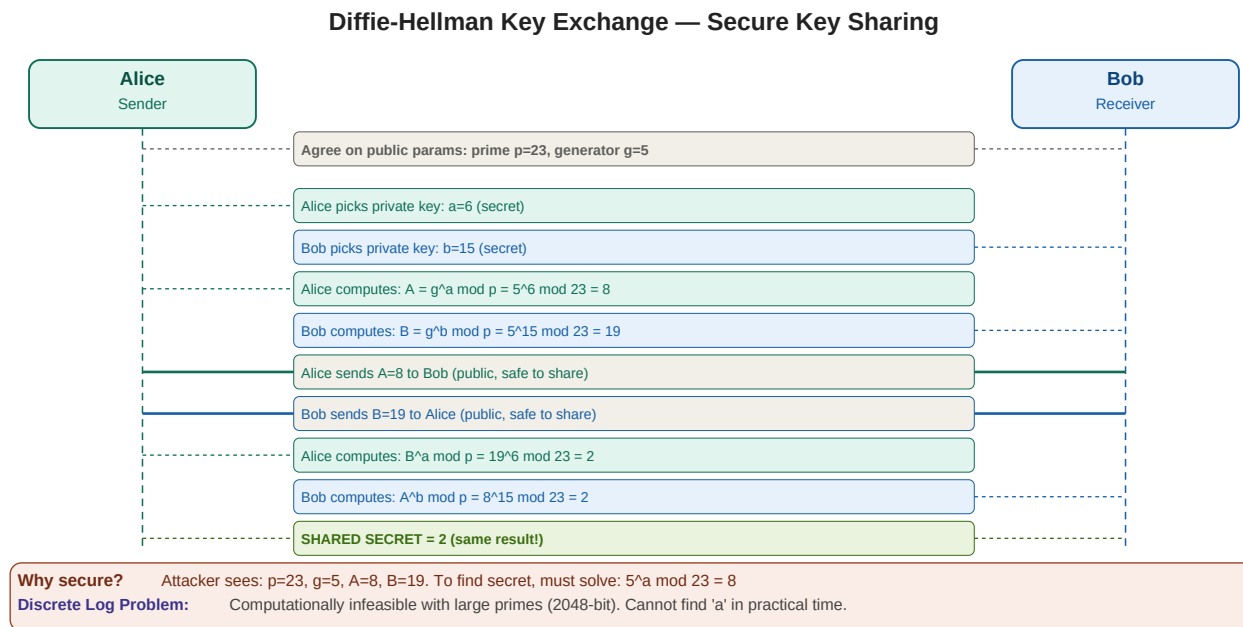


AES-128: 10 rounds · AES-192: 12 rounds · AES-256: 14 rounds · Each round: SubBytes + ShiftRows + MixColumns + AddRoundKey

AES Operation	What it does	Why it matters
SubBytes	Each byte replaced using a fixed lookup table (S-Box)	Adds non-linearity — prevents algebraic attacks
ShiftRows	Each row cyclically shifted left by its row index (0,1,2,3)	Distributes bytes across all columns
MixColumns	Matrix multiplication on each 4-byte column	Diffuses data — changes in one byte affect all
AddRoundKey	State XORed with the round key for this round	Incorporates secret key material every round
Key Expansion	128-bit key expanded to 44 words (11 round keys)	Unique key material per round — no reuse

4. Diffie-Hellman Key Exchange

Diffie-Hellman (DH) solves the fundamental problem of symmetric encryption: how do two parties agree on a shared secret key over an insecure network without ever transmitting the key itself? DH uses the mathematical hardness of the **discrete logarithm problem**.



Public params (p, g) shared openly · Private keys (a, b) never transmitted · Both compute same shared secret independently

The magic: Alice computes $B^a \bmod p = 19^6 \bmod 23 = 2$. Bob computes $A^b \bmod p = 8^{15} \bmod 23 = 2$. Both get the same answer — without ever sending their private keys!

Security: an attacker sees $p=23, g=5, A=8, B=19$ but cannot find "a" from $5^a \bmod 23 = 8$ in practical time. This is the Discrete Logarithm Problem.

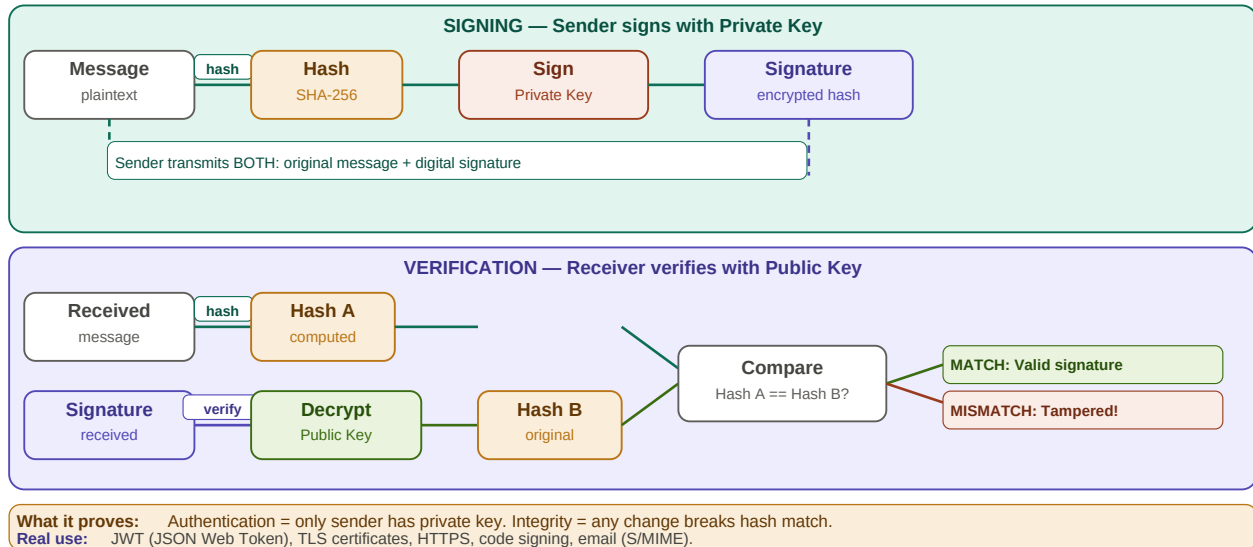
Real world: HTTPS/TLS uses DH (or ECDH) to establish a session key, then AES uses that key to encrypt all traffic. The session key changes per connection (Perfect Forward Secrecy).

ECDH (Elliptic Curve DH): same concept but uses elliptic curve math — same security with smaller keys (256-bit ECDH ~ 3072-bit classical DH).

5. Digital Signatures — Authentication & Integrity

A **digital signature** answers two questions: "Did this message really come from Alice?" (authentication) and "Was it modified in transit?" (integrity). It uses asymmetric key pairs and a hash function — the signature can be verified by anyone, but only created by the key owner.

Digital Signatures — Authentication + Integrity



Signing: private key encrypts the hash · Verification: public key decrypts hash, compare with recomputed hash · Match = valid

Step	Who	Action	Why
1. Hash	Sender	Run message through SHA-256 → fixed-size hash	Small, unique fingerprint of the message
2. Sign	Sender	Encrypt hash with private key → digital signature	Only sender can do this — proves identity
3. Send	Sender	Transmit: original message + digital signature	Both needed for verification
4. Re-hash	Receiver	Run received message through SHA-256	Independently compute expected hash
5. Verify	Receiver	Decrypt signature with sender's public key	Retrieve sender's original hash
6. Compare	Receiver	Hash A == Hash B? Valid : Tampered	Any change in message changes hash

JWT (JSON Web Token): header.payload.signature — the signature is a digital signature over header+payload using the server's private key. Anyone can verify with the public key.

Hash functions (SHA-256): one-way — cannot reverse a hash to get original data. Even 1-bit change in input produces completely different hash (avalanche effect).

TLS Certificates: websites use digital signatures from a Certificate Authority (CA) to prove their identity. Your browser verifies the certificate signature using the CA's public key.

Code signing: software packages digitally signed by publisher. OS verifies before running — confirms code is unmodified and from a trusted source.

6 & 7. Algorithm Summary & Interview Questions

Algorithm Comparison

Algorithm	Type	Key Size	Speed	Use Case	Status
AES-256	Symmetric	256-bit	Very fast	HTTPS traffic, file encryption, WhatsApp	Current standard
DES	Symmetric	56-bit	Fast	Legacy systems only	BROKEN — avoid
RSA	Asymmetric	2048-bit+	Slow	TLS handshake, certificates, key wrapping	Current standard
DSA	Asymmetric	1024-3072-bit	Medium	Digital signatures only	Being replaced by ECDSA
Diffie-Hellman	Key Exchange	2048-bit+	Slow	TLS key exchange (before AES session)	Current standard
ECDH	Key Exchange	256-bit	Fast	TLS 1.3, mobile apps, Signal protocol	Preferred modern choice
SHA-256	Hash	256-bit output	Very fast	Digital signatures, JWT, TLS certs	Current standard
MD5	Hash	128-bit output	Fast	Legacy checksums only	BROKEN — collision attacks

Quick Revision

- > Cryptography = confidentiality + integrity + authentication.
- > Symmetric: same key encrypts and decrypts. Fast. Problem: key distribution.
- > Asymmetric: public key encrypts, private key decrypts. Slow. Solves key distribution.
- > AES: block cipher. 128-bit blocks. Key sizes: 128 (10 rounds), 192 (12 rounds), 256 (14 rounds).
- > AES round operations: SubBytes (S-Box lookup), ShiftRows (row rotation), MixColumns (diffusion), AddRoundKey (XOR with round key).
- > AES Key Expansion: 128-bit key -> 44 words (11 round keys) via XOR + substitution.
- > DH: agree on public p,g. Each picks private key. Exchange $g^{\text{priv}} \bmod p$. Both compute shared secret.
- > DH security: Discrete Log Problem — cannot find private key from public values in practical time.
- > Digital signature: hash message, encrypt hash with private key = signature. Verify with public key.
- > Hash functions: one-way, fixed output, avalanche effect. SHA-256 produces 256-bit output.
- > JWT: header.payload.signature. Signature = digital signature with server private key.
- > HTTPS: DH exchanges AES session key, then AES encrypts all traffic.
- > Perfect Forward Secrecy: new DH key per session — past sessions safe even if key compromised.
- > DES (56-bit): broken by brute force. Always use AES instead.

-> ECDH vs DH: same math but elliptic curves — 256-bit ECDH = security of 3072-bit DH.

Interview Questions

- > 1. What is the difference between encryption and hashing?
- > 2. What is the difference between symmetric and asymmetric encryption?
- > 3. Why do we use both symmetric and asymmetric encryption in HTTPS?
- > 4. How does AES work? Explain the four operations in each round.
- > 5. What is the key distribution problem? How does Diffie-Hellman solve it?
- > 6. Walk me through the Diffie-Hellman key exchange step by step.
- > 7. Why can't an eavesdropper compute the DH shared secret even seeing all public values?
- > 8. What is a digital signature? How does it provide authentication and integrity?
- > 9. How is a digital signature different from encryption?
- > 10. What is a hash function? What properties must it have?
- > 11. How does JWT use digital signatures?
- > 12. What makes DES insecure? Why is AES preferred?
- > 13. What is Perfect Forward Secrecy and why does it matter?
- > 14. How does HTTPS use cryptography end to end?

Key Terms

Plaintext	Original readable data before encryption
Ciphertext	Encrypted data — unreadable without the key
Symmetric Encryption	Same key for encrypt and decrypt. Fast. AES, DES.
Asymmetric Encryption	Public key encrypts, private key decrypts. RSA, DSA.
AES	Advanced Encryption Standard. Block cipher. 128/192/256-bit keys.
DES	Data Encryption Standard. 56-bit key. Obsolete — broken.
RSA	Asymmetric algorithm based on prime factorization. 2048-bit keys.
Diffie-Hellman	Key exchange protocol. Shares symmetric key over insecure channel.
ECDH	Elliptic Curve DH. Same as DH but smaller keys. Used in TLS 1.3.
Hash Function	One-way function. Fixed output. SHA-256 produces 256-bit hash.
Digital Signature	Hash encrypted with private key. Proves authenticity + integrity.
State Array	4x4 byte matrix AES operates on each round.
S-Box	Substitution table used in AES SubBytes operation.
Key Expansion	Process of deriving multiple round keys from original AES key.
Discrete Log Problem	Mathematical hardness underlying DH security.

Perfect Forward
Secrecy

New session key each connection — past sessions protected.